

# M2 – React-Umbau & Interaktion

---

Web-Applikationen SS26 · Prof. Dr. Pascal Laube · HTWG Konstanz

Abgabe: nach VL 09 · VL-Bezug: VL 07–09 (Tooling, React, Hooks)

---

## Was ihr abgibt

- **README.md** aus dem Repository direkt in Moodle abgeben (copy-paste oder Upload)
  - Die README muss enthalten:
    - Den **Repository-Link**
    - Eine **Zuordnungstabelle**: welches Kriterium ist in welcher Datei/Zeile erfüllt
    - Dokumentation, dass `npm install` && `npm run dev` funktioniert
- 

## Kriterien

### Tooling (VL 07)

Ihr richtet ein vollständiges Frontend-Tooling ein:

- **npm** als Paketmanager, **Vite** als Bundler (empfohlen: `npm create vite@latest`)
- Wählt dabei **React + TypeScript** als Template
- **TypeScript** muss aktiv genutzt sein, also nicht nur `tsconfig.json` vorhanden:
  - Props von Komponenten sind typisiert (kein `any` als Abkürzung)
  - Eigene Interfaces oder Types für eure Datenmodelle (z.B. `interface Recipe { ... }`)
  - Mindestens in den zentralen Dateien (App, Hauptkomponenten) echte Typen sichtbar

**Woran man es sieht:** TypeScript-Fehler werden angezeigt und behoben, nicht ignoriert. Wenn ihr einfach alles mit `any` erschlägt, gilt das Kriterium als nicht erfüllt.

---

### Komponenten & Props (VL 08)

Der M1-Prototyp war eine statische HTML-Datei. Jetzt zerlegt ihr ihn in wiederverwendbare Komponenten:

- **Mindestens 3 sinnvolle Komponenten** (nicht `Header`, `Footer`, `Main` als leere Hüllen)
- Komponenten bekommen ihre Daten über **Props** und rendern darauf basierend
- Die `App.tsx` ist Orchestrierung, keine Monolith-Datei mit allem drin

**Konkretes Beispiel** für eine Rezept-App:

- `RecipeList` bekommt `recipes: Recipe[]` als Prop und rendert eine Liste
- `RecipeCard` bekommt `recipe: Recipe` und zeigt Titel, Beschreibung, Bild
- `AddRecipeForm` hat eigenen lokalen State und ruft `onAdd: (recipe: Recipe) => void` auf

**Woran man es sieht:** Wenn ihr eine Komponente in eine andere Datei verschiebt und sie per Props konfigurieren kann, ohne die Elternkomponente zu ändern, ist das der richtige Weg.

---

## State & Hooks (VL 09)

Die App ist jetzt interaktiv, nicht nur statisch dargestellt:

- **useState** für lokalen Zustand, z.B.:
  - Eine Liste von Einträgen, die sich verändert
  - Formulareingaben (controlled components)
  - Toggle-Zustände (Menü offen/zu, Darkmode)
- **useEffect** wo es Sinn ergibt, z.B.:
  - Daten aus **localStorage** beim ersten Render laden
  - Einen externen Wert subscriben und beim Unmount aufräumen
  - (Hinweis: echtes API-Fetching kommt erst in M3 — **useEffect** für Fetch ist hier optional, aber erlaubt)

**Mindestens eine durchgängige Nutzeraktion** muss demonstrierbar sein. Konkret bedeutet das: der Nutzer tut etwas, und die UI verändert sich sichtbar und sinnvoll. Beispiele:

| Nutzeraktion                    | Sichtbare Reaktion                        |
|---------------------------------|-------------------------------------------|
| Formular ausfüllen und absenden | Neuer Eintrag erscheint in der Liste      |
| Suchfeld befüllen               | Liste filtert sich live                   |
| Kategorie auswählen             | Nur passende Einträge werden angezeigt    |
| Mehrstufiger Wizard             | "Weiter"-Klick zeigt den nächsten Schritt |

**Woran man es sieht:** Man kann die App im Browser öffnen, etwas eingeben oder klicken, und die Seite reagiert ohne Reload.

---

## Ergebnis

Der HTML/CSS-Prototyp aus M1 lebt jetzt als **React/TypeScript-App**. Die App zeigt echte Interaktion, kein leeres Gerüst.

---

## README-Vorlage für M2

```
# Projektname

**Team:** Vorname Nachname (Matrikelnummer), Vorname Nachname (Matrikelnummer)
**Repository:** https://github.com/...

## Setup

```bash
npm install
npm run dev
```

Die App läuft dann unter <http://localhost:5173>.

## Kriterien-Zuordnung M2

| Kriterium                 | Datei                                          | Zeile / Hinweis                             |
|---------------------------|------------------------------------------------|---------------------------------------------|
| npm + Vite                | package.json, vite.config.ts                   | Projekt-Root                                |
| TypeScript aktiv genutzt  | src/types.ts,<br>src/components/RecipeCard.tsx | Z. 1–15 (Interface), Z. 8 (Props)           |
| Komponentenzerlegung      | src/components/                                | RecipeList, RecipeCard,<br>AddRecipeForm    |
| Props-Übergabe            | src/App.tsx                                    | Z. 30–45                                    |
| useState                  | src/App.tsx                                    | Z. 12 (recipes-State), Z. 18 (filter-State) |
| useEffect                 | src/App.tsx                                    | Z. 22 (localStorage laden)                  |
| Durchgängige Nutzeraktion | src/components/AddRecipeForm.tsx               | Formular → Liste                            |

---

### ## Abgabe-Checkliste

- [ ] README in Moodle abgegeben (copy-paste oder Upload)
- [ ] README enthält den Repository-Link
- [ ] README enthält Zuordnungstabelle: welches Kriterium in welcher Datei/Zeile
- [ ] Beide Teammitglieder haben Commits beigetragen
- [ ] ``npm install && npm run dev`` ist dokumentiert und funktioniert
- [ ] TypeScript aktiv genutzt (Interfaces/Types vorhanden, nicht nur ``any``)
- [ ] Mindestens 3 sinnvolle Komponenten
- [ ] Props-Übergabe sichtbar
- [ ] ``useState`` in Verwendung
- [ ] Durchgängige Nutzeraktion demonstrierbar

---

### ## Tipps

- Startet mit ``npm create vite@latest`` und wählt React + TypeScript.
- Fangt mit einer Komponente an, die ihr aus M1 kennt (z.B. die Listenansicht).
- Definiert zuerst euer Datenmodell als TypeScript-Interface, bevor ihr Komponenten baut. Das hilft enorm.
- TypeScript-Fehler sind keine Fehler, sondern Hinweise. Lest sie.
- ``useEffect`` braucht ein leeres Dependency-Array ``[]``, wenn es nur einmal beim Mount laufen soll.
- Keine Secrets im Repository.

